

Математически поглед: Loss и градиент

Как моделът разбира грешката си?

Лекция 2 · Блок 3

Нашата верига

$$\mathbf{x} \xrightarrow{W, \mathbf{b}} \mathbf{z} \xrightarrow{\text{softmax}} \mathbf{P} \xrightarrow{-\log P_y} L$$

Символ	Значение	Размер
$\mathbf{x}$	one-hot вход	$\mathbb{R}^V$
$W, \mathbf{b}$	тегла и отместване	$\mathbb{R}^{V \times V}, \mathbb{R}^V$
$\mathbf{z} = \mathbf{x}W + \mathbf{b}$	логити	$\mathbb{R}^V$
$P_j = e^{z_j} / \sum_k e^{z_k}$	вероятности	$\mathbb{R}^V$
$L = -\log P_y$	loss (y = верен символ)	скалар

Въпросът: как промяната на W влияе на L ?

Защо $-\log(P)$? Интуиция

$-\log(P)$ измерва **изненадата**: колко неочаквано е събитие с вероятност P .

- $P = 0.99 \Rightarrow -\log(0.99) \approx 0.01$ — почти сигурен, нулева изненада
- $P = 0.50 \Rightarrow -\log(0.50) \approx 0.69$ — умерена изненада
- $P = 0.01 \Rightarrow -\log(0.01) \approx 4.60$ — огромна изненада

Защо не просто $1 - P$? Защото $-\log$ наказва *уверено грешните* предсказания непропорционално силно. Модел, който казва „99% сигурен съм“ и греши, получава loss 4.6 — не 0.99.

Защо точно $\log$? Единствената функция, при която loss на независими събития се *събира*:

$$-\log(P_1 \cdot P_2) = -\log P_1 + (-\log P_2)$$

Лог превръща произведения на вероятности в суми на loss стойности — затова loss-ът на цяла партида е просто средното на отделните.

Cross-entropy: обща формула

За истинско разпределение $\mathbf{q}$ и предсказано $\mathbf{p}$ върху C класа:

$$H(\mathbf{q}, \mathbf{p}) = - \sum_{j=1}^C q_j \log p_j$$

Измерва колко добре $\mathbf{p}$ описва $\mathbf{q}$ — колко средна „изненада“ получаваме, ако реалността е $\mathbf{q}$, а ние кодираме с $\mathbf{p}$.

- Ако $\mathbf{p} = \mathbf{q} \rightarrow$ минимална изненада (равна на ентропията $H(\mathbf{q})$)
- Ако $\mathbf{p}$ се различава от $\mathbf{q} \rightarrow$ повече изненада $\rightarrow$ по-голям loss

Cross-entropy при класификация

При класификация с one-hot етикети $\mathbf{q} = \mathbf{e}_y$ (единица на позиция y , нули другаде):

$$H(\mathbf{e}_y, \mathbf{P}) = - \sum_j 1[j = y] \cdot \log P_j = -\log P_y$$

Всички членове освен $j = y$ изчезват — остава познатият ни $-\log P_y$.

За партида от N примера: $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (-\log P_{y_i})$

Softmax + cross-entropy — не е случайна комбинация:

- Softmax превръща логити във валидно вероятностно разпределение
- Cross-entropy измерва колко далеч е то от идеалното
- Заедно, градиентът им се опростява до $\mathbf{P} - \mathbf{e}_y$ (ще видим!)

Простотата на градиента е причината тази двойка да стане стандарт.

Целта: $\frac{\partial L}{\partial W}$

Искаме за всяко тегло W_{kj} :

„Ако увелича W_{kj} с малко ϵ , с колко ще се промени L ?“

Верижно правило (chain rule):

$$\frac{\partial L}{\partial W_{kj}} = \underbrace{\frac{\partial L}{\partial z_j}}_{\text{търсим това}} \cdot \underbrace{\frac{\partial z_j}{\partial W_{kj}}}_{=x_k}$$

Тъй като $z_j = \sum_k x_k W_{kj} + b_j$, производната по W_{kj} е просто x_k .

Остава само да намерим $\frac{\partial L}{\partial z_j}$.

Разписваме L директно

$$L = -\log P_y = -\log \frac{e^{z_y}}{\sum_k e^{z_k}} = -z_y + \log \sum_k e^{z_k}$$

Вземаме производна по z_j :

$$\frac{\partial L}{\partial z_j} = -\frac{\partial z_y}{\partial z_j} + \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Забелязваме, че $\frac{e^{z_j}}{\sum_k e^{z_k}} = P_j$.

Два случая

Случай 1: $j \neq y$ (грешен символ)

$$\frac{\partial z_y}{\partial z_j} = 0 \quad \Rightarrow \quad \frac{\partial L}{\partial z_j} = P_j$$

Случай 2: $j = y$ (верният символ)

$$\frac{\partial z_y}{\partial z_j} = 1 \quad \Rightarrow \quad \frac{\partial L}{\partial z_j} = -1 + P_y = P_y - 1$$

Двата случая в едно:

$$\frac{\partial L}{\partial z_j} = P_j - 1[j = y]$$

Красивият резултат

Векторно — за всички логити наведнъж:

$$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{P} - \mathbf{e}_y$$

Където $\mathbf{e}_y$ е вектор с 1 на позиция y и 0 навсякъде другаде.

Интуиция: Градиентът е просто разликата между предсказаните и верните вероятности.

Не е нужна сложна математика — само извадена.

Точно това обещахме: softmax + cross-entropy дават максимално прост градиент.

От $\frac{\partial L}{\partial \mathbf{z}}$ към $\frac{\partial L}{\partial W}$

$$\frac{\partial L}{\partial W_{kj}} = \frac{\partial L}{\partial z_j} \cdot x_k = (P_j - 1[j = y]) \cdot x_k$$

Матрично (за всички тегла наведнъж):

$$\boxed{\frac{\partial L}{\partial W} = \mathbf{x}^\top \cdot \frac{\partial L}{\partial \mathbf{z}}} \in \mathbb{R}^{V \times V}$$

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{z}} \in \mathbb{R}^V$$

One-hot трик: само един ред се обновява

Тъй като $\mathbf{x}$ е one-hot (само $x_i = 1$, всичко друго 0):

$$\frac{\partial L}{\partial W_{kj}} = (P_j - 1[j = y]) \cdot x_k = \begin{cases} P_j - 1[j = y] & k = i \\ 0 & k \neq i \end{cases}$$

Само ред i (редът на входния символ) получава ненулев градиент:

$$\frac{\partial L}{\partial W_{i,\cdot}} = \mathbf{P} - \mathbf{e}_y$$

Оттук и ефективността на `nn.Embedding` — вместо да умножаваме по $\mathbf{x}$, директно вземаме ред i от W .

Пример с числа ($V = 4$)

Речник: ' ' (0), 'а' (1), 'б' (2), 'в' (3). Вход: 'а' ($i = 1$), цел: 'в' ($y = 3$), $\eta = 0.1$.

Forward:

$$\mathbf{x} = [0, 1, 0, 0] \rightarrow \mathbf{z} = [2.0, 0.0, 1.0, 0.5] \rightarrow \mathbf{P} = [0.58, 0.08, 0.21, 0.13] \rightarrow L \approx 2.04$$

Backward:

$$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{P} - \mathbf{e}_3 = [+0.58, +0.08, +0.21, -0.87]$$

Update (one-hot $\rightarrow$ само ред 1 на W):

$$W_{1,\cdot} \leftarrow [2.0, 0.0, 1.0, 0.5] - 0.1 \times [0.58, 0.08, 0.21, -0.87] = [1.94, -0.01, 0.98, \mathbf{0.59}]$$

Логитът на 'в' нарасна от 0.5 $\rightarrow$ 0.59 — моделът се е научил малко!

Обобщение

$$\mathbf{x} \xrightarrow{W, \mathbf{b}} \mathbf{z} \xrightarrow{\text{softmax}} \mathbf{P} \xrightarrow{-\log P_y} L$$

Градиент	Формула	Размер
$\frac{\partial L}{\partial \mathbf{z}}$	$\mathbf{P} - \mathbf{e}_y$	$\mathbb{R}^V$
$\frac{\partial L}{\partial W}$	$\mathbf{x}^\top (\mathbf{P} - \mathbf{e}_y)$	$\mathbb{R}^{V \times V}$
$\frac{\partial L}{\partial \mathbf{b}}$	$\mathbf{P} - \mathbf{e}_y$	$\mathbb{R}^V$

Ключова интуиция: Градиентът „наказва“ вероятностите на грешните символи и „награждава“ верния.

Следва: как тези градиенти текат *назад* през *дълбока* мрежа → [Backpropagation]